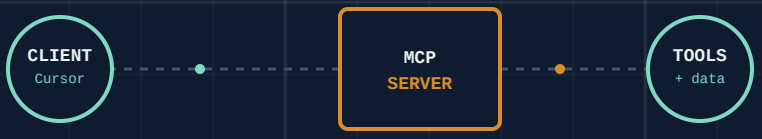
Transport: stdio / http+sse
Status: production



// A NODE & TYPESCRIPT FIELD GUIDE

Shipping Production MCP Servers

Auth, rate limiting, structured errors, testing, and real deployment — everything the free templates leave out. Written for developers who already know their way around Node and Express.

MCP Server Starter Pack

BUILD · TEST · SHIP

TYPESCRIPT NODE.JS

// A FIELD GUIDE FOR NODE & TYPESCRIPT DEVELOPERS

Shipping Production MCP Servers

Everything the free templates skip: authentication, rate limiting, structured error handling, testing, containerization, and real deployment — built from the same instincts you already use for Express APIs and MERN backends.

MCP Server Starter Pack

COMPANION GUIDE · BUILD, TEST, SHIP

Contents

// 14 CHAPTERS · 2 APPENDICES

FRONT MATTER

– Foreword: Why MCP, Why Now 3

PART I — FOUNDATIONS

01 MCP Fundamentals 4

02 Project Foundation 8

03 Designing Tools with Zod 12

04 Transports: stdio and HTTP/SSE 15

PART II — PRODUCTION INFRASTRUCTURE

05 Authentication & Authorization 18

06 Rate Limiting 21

07 Error Handling & Structured Logging 24

08 Testing & the MCP Inspector 27

PART III — SHIPPING

09 Containerizing with Docker 30

10 Deployment: Railway, Fly.io & Serverless 33

11 CI/CD with GitHub Actions 36

PART IV — REAL TOOLS & POLISH

12 Real-World Integration Examples 39

13 Developer Experience Polish 43

14 The Shipping Checklist 46

APPENDICES

A Quick-Reference Cheat Sheet 49

B Further Reading & Resources 52

FOREWORD

Why MCP, Why Now

If you've spent the last few years writing Express routes, wiring up REST APIs, and shaping request/response contracts for a MERN stack, you already have almost everything you need to build a Model Context Protocol server. The remaining gap isn't a new language or a new mental model — it's a handful of new nouns (tools, resources, prompts) sitting on top of infrastructure you've built a dozen times before.

MCP is the protocol that lets AI assistants — Claude Desktop, Cursor, VS Code Copilot, and a fast-growing list of others — call out to external tools, read from external resources, and use reusable prompts, all through one standard interface. Instead of every AI product inventing its own plugin format, MCP gives developers one server to write that works everywhere the protocol is supported.

That adoption curve is exactly why the timing matters. When a protocol moves this fast, the tooling around it lags behind the demand for real, production-grade implementations. Search for an MCP starter template today and you'll find dozens of repositories — most of them a single `tools/list` handler and a "hello world" tool, with no auth, no rate limiting, no deployment story, and no tests. They're fine for a five-minute demo. They fall apart the moment you want to hand a server to a teammate, put it behind a real endpoint, or charge someone money for access to it.

This guide closes that gap. It walks through building an MCP server the way you'd actually want to run it in production: with schema-validated inputs, authenticated requests, rate-limited tools, structured error handling that doesn't leak stack traces to a client, a real test suite, and deploy configs for more than one hosting target. Every chapter pairs the concept with working TypeScript, because the fastest way to understand a new piece of infrastructure is to read the fifteen lines of code that implement it — not a paragraph of abstract description.

You don't need to learn a new stack to use this book. You need to point the Node and TypeScript instincts you already have at a new target.

// CHAPTER 01 / PART I

MCP Fundamentals

Before writing a line of production infrastructure, get the mental model right: what MCP actually standardizes, the three primitives every server is built from, and how a client and server actually talk to each other.

THIS CHAPTER COVERS

- The problem MCP standardizes
- Tools, resources, and prompts
- The initialize → list → call lifecycle
- Transport options at a glance

CHAPTER 01

MCP Fundamentals

The problem MCP standardizes

Before MCP, every AI product that wanted to call external tools invented its own integration format. A plugin written for one assistant didn't work in another; a developer supporting three AI products maintained three separate integration layers that did roughly the same thing. The Model Context Protocol replaces that with one open specification: a client (the AI application — Claude Desktop, Cursor, an IDE extension) speaks a standard JSON-RPC-based protocol to a server (your code), and any MCP-compliant client can use any MCP-compliant server without custom glue.

That's the whole pitch, and it's the same pitch you've heard before for REST, for GraphQL, for any standard interface that trades a little flexibility for universal compatibility. If you've built an internal API that three different frontends consume, you already understand the value of the contract mattering more than the implementation behind it.

The three primitives

An MCP server exposes capabilities through three primitive types. Almost every real server is some combination of the three:

PRIMITIVE	WHAT IT IS	REST/EXPRESS ANALOGY
Tools	Functions the model can call, with typed inputs and a return value. The model decides when to call them based on the user's request.	A POST endpoint with a validated request body
Resources	Read-only data the client can fetch — files, database records, API responses — addressed by URI.	A GET endpoint returning a representation
Prompts	Reusable, parameterized prompt templates the client can surface to the user as a shortcut.	A saved query template

Most of the engineering effort in a production server — and most of this book — is about **tools**: validating their inputs, authenticating who can call them, rate limiting how often they're

called, and handling failures cleanly. Resources and prompts follow the same infrastructure patterns once tools are solid.

THE MESSAGE LIFECYCLE

A client and server go through the same three phases regardless of what the server actually does:

The client opens a connection and sends `initialize`, exchanging protocol versions and capabilities. It then calls `tools/list` (and `resources/list`, `prompts/list`) to discover what the server offers. When the model decides to use a tool, the client sends `tools/call` with the tool name and arguments, and your handler runs and returns a result. Every tool call in this book maps onto that same final step — the interesting engineering happens in what runs between receiving the call and returning the result.

A minimal server, end to end

Here's the smallest complete server — one tool, no infrastructure yet. Everything in Part II adds a layer on top of this shape without changing it:

SRC/INDEX.TS

```ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({ name: "weather-mcp", version: "1.0.0" });

server.tool(
  "get_forecast",
  "Get the weather forecast for a city",
  { city: z.string().describe("City name, e.g. 'Lisbon'") },
  async ({ city }) => {
    const forecast = await fetchForecast(city); // your own fetch logic
    return { content: [{ type: "text", text: forecast }] };
  }
);

const transport = new StdioServerTransport();
await server.connect(transport);
```

> **WHERE YOUR INSTINCTS TRANSFER**
>
> The `z.string().describe(...)` call is doing double duty familiar from any Express + Zod middleware you've written: it validates the shape of the input *and* documents it for the model, since the description gets surfaced to the client during `tools/list`. Type-safe input validation isn't new territory — it's the same discipline you already apply to request bodies, pointed at a different transport.

Transports, briefly

MCP servers run over one of two transports, covered in full in Chapter 4: **stdio**, where the client spawns your server as a subprocess and talks over standard input/output (this is how Claude Desktop and most local IDE integrations work), and **HTTP with Server-Sent Events**, where your server runs as a long-lived process reachable over the network — the shape you'd choose for a hosted, multi-user server. A production starter kit supports both from the same tool definitions, switching only the transport layer underneath.

// CHAPTER 02 / PART I

Project Foundation

Set up the project once, correctly, and every chapter after this one slots into a folder that already knows where things go.

THIS CHAPTER COVERS

- Strict TypeScript configuration
- A folder structure that scales
- Environment & secrets management
- The config module pattern

CHAPTER 02

Project Foundation

Strict TypeScript from day one

Turn on strict mode before you write the first tool, not after the fifth one ships a bug that `strict: true` would have caught. The cost of retrofitting strictness onto a codebase grows with every file you add; the cost of starting strict is a handful of extra type annotations up front.

TSCONFIG.JSON

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "outDir": "dist",
    "rootDir": "src",
    "declaration": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*"]
}
```

`noUncheckedIndexedAccess` and `exactOptionalPropertyTypes` are the two settings most Express projects skip and most MCP servers need: tool arguments arrive as loosely-typed JSON before Zod narrows them, and an index signature that silently returns `undefined` is exactly the class of bug that turns into a cryptic error three layers deep in a tool handler.

A folder structure that scales

The structure below is deliberately close to a well-organized Express API — routes become tools, middleware stays middleware — so nothing here should require relearning how you already split concerns.

```
PROJECT STRUCTURE

src/
├── index.ts              # entrypoint, transport wiring
├── server.ts             # McpServer instance + registration
├── config/
│   └── env.ts             # validated environment config
├── tools/
│   ├── index.ts           # registers every tool
│   ├── get-forecast.ts
│   └── query-database.ts
├── resources/
│   └── index.ts
├── prompts/
│   └── index.ts
├── middleware/
│   ├── auth.ts
│   ├── rate-limit.ts
│   └── error-handler.ts
└── lib/
    ├── logger.ts
    └── errors.ts
```

Each tool lives in its own file and exports a single registration function. That keeps `tools/index.ts` as a manifest — a place to see every capability the server exposes at a glance — the same way a well-kept `routes/index.ts` works in an Express app.

```
SRC/TOOLS/INDEX.TS

import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { registerGetForecast } from "./get-forecast.js";
import { registerQueryDatabase } from "./query-database.js";

export function registerAllTools(server: McpServer) {
  registerGetForecast(server);
  registerQueryDatabase(server);
}
```

Environment & secrets management

Validate environment variables at startup the same way you validate tool inputs — with a schema, not with scattered `process.env.X!` assertions that fail silently in production. A server that crashes immediately with a clear message because `DATABASE_URL` is missing is dramatically easier to operate than one that starts fine and fails on the first tool call twenty minutes later.

SRC/CONFIG/ENV.TS

```ts
import { z } from "zod";

const EnvSchema = z.object({
  NODE_ENV: z.enum(["development", "production", "test"]).default("development"),
  PORT: z.coerce.number().default(3000),
  API_KEYS: z.string().min(1, "API_KEYS must contain at least one key"),
  DATABASE_URL: z.string().url().optional(),
  RATE_LIMIT_PER_MINUTE: z.coerce.number().default(60),
  LOG_LEVEL: z.enum(["debug", "info", "warn", "error"]).default("info"),
});

export const env = EnvSchema.parse(process.env);
// Throws a readable ZodError at boot if anything required is missing —
// not three tool calls later.
```

SECRETS PATTERN

Keep a checked-in `.env.example` with every key name and a placeholder value, and add `.env` to `.gitignore` on day one. For deployed servers, set the real values as platform secrets (Railway/Fly.io environment variables, GitHub Actions encrypted secrets for CI) — never as a committed file, even in a private repo.

// CHAPTER 03 / PART I

Designing Tools with Zod

The quality of your tool schemas determines whether the model calls your tools correctly on the first try — this is prompt engineering and API design at the same time.

THIS CHAPTER COVERS

- Schemas as documentation
- Enums, unions & nested objects
- Reusable schema fragments
- Returning structured errors

CHAPTER 03

Designing Tools with Zod

A schema is documentation the model reads

Every `.describe()` call on a Zod schema becomes part of what the model sees when deciding whether and how to call your tool. This is a genuinely different constraint from validating an Express request body: there, the schema only needs to be correct. Here, it also needs to be legible to a model that has never seen your codebase and is inferring intent from field names and descriptions alone.

SRC/TOOLS/QUERY-DATABASE.TS

```ts
import { z } from "zod";

const QueryInput = z.object({
  table: z.enum(["users", "orders", "products"])
    .describe("Which table to query"),
  filters: z.record(z.string(), z.union([z.string(), z.number()]))
    .optional()
    .describe("Column-to-value equality filters, e.g. { status: 'active' }"),
  limit: z.number().int().min(1).max(100).default(20)
    .describe("Max rows to return (1-100)"),
});

type QueryInput = z.infer<typeof QueryInput>;
```

Notice the same instinct at work that you'd apply to a well-designed REST query parameter: constrain the shape as tightly as the domain allows (`enum` instead of free-text `string` for `table`), bound anything numeric, and default anything optional to a sane value rather than leaving it undefined for the handler to guess at.

REUSABLE SCHEMA FRAGMENTS

As the tool count grows, factor out fragments that repeat — pagination, date ranges, common ID formats — into a shared module, exactly like you'd extract a shared Joi/Zod validator across Express routes.

SRC/TOOLS/SHARED-SCHEMAS.TS

```ts
export const Pagination = z.object({
  page: z.number().int().min(1).default(1),
  pageSize: z.number().int().min(1).max(100).default(25),
});

export const DateRange = z.object({
  from: z.string().datetime().describe("ISO 8601 start date"),
  to: z.string().datetime().describe("ISO 8601 end date"),
}).refine(r => new Date(r.from) <= new Date(r.to), {
  message: "'from' must be before 'to'",
});
```

Returning structured, model-readable errors

When a tool fails validation or hits a business-logic error, resist the urge to throw a raw exception. Return a structured error result the model can actually reason about — the MCP equivalent of returning a `400` with a clear JSON body instead of letting Express's default error handler dump a stack trace.

SRC/TOOLS/QUERY-DATABASE.TS (EXCERPT)

```ts
server.tool("query_database", "Query rows from an allowed table", QueryInput.shape,
  async (input) => {
    const parsed = QueryInput.safeParse(input);
    if (!parsed.success) {
      return {
        isError: true,
        content: [{ type: "text", text: `Invalid input: ${parsed.error.message}` }],
      };
    }
    try {
      const rows = await runQuery(parsed.data);
      return { content: [{ type: "text", text: JSON.stringify(rows) }] };
    } catch (err) {
      return {
        isError: true,
        content: [{ type: "text", text: "Query failed. Check table name and filters." }],
      };
    }
  }
);
```

> **DON'T LEAK INTERNALS**
>
> The `catch` block above deliberately returns a generic message instead of `err.message`. Database errors often contain table names, column names, or query fragments — treat them the same way you'd treat an error you're about to send to an untrusted HTTP client. Full detail goes to your logger (Chapter 7), not to the model.

// CHAPTER 04 / PART I

Transports: stdio & HTTP/SSE

Wire up both transport modes from the same tool registration code, so the identical server runs locally under Claude Desktop and remotely as a hosted, multi-user endpoint.

THIS CHAPTER COVERS

- stdio for local clients
- HTTP + SSE for remote servers
- Session management
- Sharing one server across both

CHAPTER 04

Transports: stdio and HTTP/SSE

Two transports, one server

The trick to supporting both transports cleanly is the same trick you already use to share business logic between a CLI script and an HTTP route: keep tool registration transport-agnostic, and only branch at the very edge of the application.

SRC/SERVER.TS

```ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { registerAllTools } from "./tools/index.js";
import { registerAllResources } from "./resources/index.js";

export function buildServer(): McpServer {
  const server = new McpServer({ name: "mcp-starter", version: "1.0.0" });
  registerAllTools(server);
  registerAllResources(server);
  return server;
}
```

STDIO — FOR LOCAL CLIENTS

Claude Desktop, Cursor, and most IDE integrations spawn your server as a child process and speak MCP over its stdin/stdout. There's no networking, no auth layer to worry about (the OS process boundary is the trust boundary), and no concurrent-session handling — one client, one process.

SRC/INDEX.STDIO.TS

```ts
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { buildServer } from "./server.js";

const server = buildServer();
const transport = new StdioServerTransport();
await server.connect(transport);
// Never console.log() here — stdout is the protocol channel.
// Use the logger from Chapter 7, which writes to stderr instead.
```

> **THE #1 STDIO BUG**
>
> A stray `console.log()` anywhere in your dependency tree corrupts the protocol stream on stdio, because stdout is the wire, not a debug console. Route every log line to stderr, and treat any library that logs to stdout by default as one you need to configure or replace.

HTTP + SSE — FOR HOSTED, MULTI-USER SERVERS

A remote server needs to handle multiple concurrent client sessions, which means transport setup looks like standard Express middleware — because it is Express middleware, with a session-aware transport in front of it.

SRC/INDEX.HTTP.TS

```
import express from "express";
import { randomUUID } from "node:crypto";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/streamableHttp.js";
import { buildServer } from "./server.js";
import { apiKeyAuth } from "./middleware/auth.js";
import { rateLimit } from "./middleware/rate-limit.js";
import { env } from "./config/env.js";

const app = express();
app.use(express.json());

const sessions = new Map<string, StreamableHTTPServerTransport>();

app.post("/mcp", apiKeyAuth, rateLimit, async (req, res) => {
  const sessionId = req.headers["mcp-session-id"] as string ?? randomUUID();
  let transport = sessions.get(sessionId);

  if (!transport) {
    transport = new StreamableHTTPServerTransport({ sessionIdGenerator: () => sessionId });
    const server = buildServer();
    await server.connect(transport);
    sessions.set(sessionId, transport);
  }

  await transport.handleRequest(req, res, req.body);
});

app.listen(env.PORT, () => logger.info(`MCP server listening on :${env.PORT}`));
```

This is the exact shape of an authenticated, rate-limited Express route — because that's precisely what it is. Chapters 5 and 6 fill in `apiKeyAuth` and `rateLimit` as ordinary Express middleware that happens to sit in front of an MCP transport instead of a JSON API route.

TRANSPORT	USE WHEN	AUTH MODEL
stdio	Local tools — Claude Desktop, Cursor, VS Code	OS process boundary; no network auth needed
HTTP + SSE	Hosted, remote, multi-user servers	API keys or OAuth, same as any public API

// CHAPTER 05 / PART II

Authentication & Authorization

API-key auth for the common case, plus an OAuth scaffold for when a hosted server needs to act on behalf of a specific user's account.

THIS CHAPTER COVERS

- API key middleware
- Per-key scopes & permissions
- OAuth 2.1 scaffold for remote servers
- Where credentials should never live

CHAPTER 05

Authentication & Authorization

API key middleware

For most MCP servers — internal tools, single-tenant integrations — an API key is the right amount of auth: simple to issue, simple to rotate, simple for a client to configure. Compare keys with a constant-time check, never `===`, to avoid leaking key material through timing differences — the same discipline you'd apply to comparing a webhook signature.

SRC/MIDDLEWARE/AUTH.TS

```
import { timingSafeEqual } from "node:crypto";
import type { Request, Response, NextFunction } from "express";
import { env } from "../config/env.js";

const validKeys = new Set(env.API_KEYS.split(",").map(k => k.trim()));

function safeCompare(a: string, b: string): boolean {
  const bufA = Buffer.from(a);
  const bufB = Buffer.from(b);
  if (bufA.length !== bufB.length) return false;
  return timingSafeEqual(bufA, bufB);
}

export function apiKeyAuth(req: Request, res: Response, next: NextFunction) {
  const header = req.headers["authorization"];
  const key = typeof header === "string" ? header.replace("Bearer ", "") : "";

  const matched = [...validKeys].some(k => safeCompare(k, key));
  if (!matched) {
    return res.status(401).json({ error: "invalid_api_key" });
  }
  next();
}
```

PER-KEY SCOPES

Once you have more than one integration consuming the same server, flat "valid or not" auth stops being enough — a read-only reporting key shouldn't be able to call a tool that writes to your database. Attach scopes to each key and check them per tool, mirroring how you'd gate an Express route behind `requireRole()`.

```
SRC/MIDDLEWARE/AUTH.TS (EXCERPT)
interface ApiKeyRecord { key: string; scopes: string[]; }

export function requireScope(scope: string) {
  return function (req: Request, res: Response, next: NextFunction) {
    const record = req.apiKeyRecord as ApiKeyRecord | undefined;
    if (!record?.scopes.includes(scope)) {
      return res.status(403).json({ error: "insufficient_scope", required: scope });
    }
    next();
  };
}
```

OAuth 2.1 scaffold

When a server needs to act on behalf of a specific end user against a third-party API — a Gmail or GitHub integration, for instance — API keys aren't the right model. The MCP spec's authorization flow is built on OAuth 2.1 with PKCE, which will look familiar if you've implemented "Sign in with Google" for a MERN app: an authorization endpoint, a token endpoint, and short-lived access tokens with refresh.

STEP	WHAT HAPPENS
1. Discovery	Client fetches `/.well-known/oauth-authorization-server`
2. Authorize	User redirected to consent screen with PKCE challenge
3. Token exchange	Client trades auth code + verifier for access/refresh tokens
4. Authenticated calls	Access token sent as a Bearer header on every `tools/call`

> **SCAFFOLD, NOT FULL IMPLEMENTATION**
>
> The starter kit ships the route shapes and PKCE verification logic wired up; you plug in your identity provider (Auth0, Clerk, or a hand-rolled provider) the same way you would for any other OAuth-backed Express app. This is the piece most templates skip entirely, and the piece a remote, user-scoped MCP server can't ship without.

// CHAPTER 06 / PART II

Rate Limiting

A model can call a tool far more aggressively than a human clicking a UI ever would. Rate limiting isn't optional polish here — it's what keeps one runaway agent loop from taking down your database.

THIS CHAPTER COVERS

- Why agent traffic is bursty
- Sliding-window limiting
- Per-tool, per-key limits
- Responding correctly when limited

CHAPTER 06

Rate Limiting

Why this matters more than it does for a typical API

A human using a web UI is naturally rate-limited by how fast they can click. A model in an agentic loop is not — it can call `query_database` fifty times in a two-second span while working through a multi-step task, especially if it's retrying after a validation error. Without limits, that pattern turns into a self-inflicted denial-of-service against your own database or a third-party API you're wrapping.

Sliding-window limiting per key

A fixed window (reset every 60 seconds on the clock) allows bursts right at the window boundary; a sliding window smooths that out with a small amount of extra bookkeeping — the same trade-off you'd weigh choosing a rate limiter for a public API.

SRC/MIDDLEWARE/RATE-LIMIT.TS

```
import type { Request, Response, NextFunction } from "express";
import { env } from "../config/env.js";

const windowMs = 60_000;
const hits = new Map<string, number[]>();

export function rateLimit(req: Request, res: Response, next: NextFunction) {
  const key = req.apiKeyRecord?.key ?? req.ip ?? "anonymous";
  const now = Date.now();
  const windowStart = now - windowMs;

  const timestamps = (hits.get(key) ?? []).filter(t => t > windowStart);
  timestamps.push(now);
  hits.set(key, timestamps);

  if (timestamps.length > env.RATE_LIMIT_PER_MINUTE) {
    res.setHeader("Retry-After", "60");
    return res.status(429).json({
      error: "rate_limited",
      message: `Limit of ${env.RATE_LIMIT_PER_MINUTE}/min exceeded`,
    });
  }
  next();
}
```

> **IN-MEMORY LIMITS DON'T SURVIVE MULTIPLE INSTANCES**
>
> The `Map` above works for a single process. The moment you scale a deployment horizontally (Chapter 10), move this state to Redis with `ZADD` / `ZREMRANGEBYSCORE` for the same sliding-window behavior shared across instances — the identical migration path you'd take for any Express rate limiter under real traffic.

Per-tool limits

Not every tool deserves the same budget. A cheap, cached lookup tool can tolerate a much higher call rate than a tool that hits a metered third-party API or writes to a database. Configure limits per tool, not just per key:

SRC/CONFIG/RATE-LIMITS.TS

```ts
export const toolLimits: Record<string, number> = {
  get_forecast: 120,      // cheap, cached
  query_database: 30,      // moderate cost
  send_email: 5,           // expensive / side-effecting
};
```

Wrap each tool handler with a check against its own budget before running the underlying logic, and return the same structured `429`-equivalent tool result described in Chapter 3 so the model can back off and retry rather than treating the failure as a hard error.

// CHAPTER 07 / PART II

Error Handling & Logging

Debuggable failures instead of silent ones: structured error types, redacted logs, and a clear line between what the model sees and what your logs capture.

THIS CHAPTER COVERS

- A structured error hierarchy
- Logging with Pino
- Redacting secrets automatically
- Correlating a call across logs

CHAPTER 07

Error Handling & Structured Logging

A small hierarchy of error types

Distinguish errors the same way you would in any well-structured Node service: errors that are the caller's fault (bad input), errors that are a downstream dependency's fault (a database timeout), and errors that are genuinely unexpected. Each deserves a different response and a different log severity.

SRC/LIB/ERRORS.TS

```
export class ToolInputError extends Error {
  constructor(message: string, public readonly field?: string) {
    super(message);
    this.name = "ToolInputError";
  }
}

export class UpstreamServiceError extends Error {
  constructor(message: string, public readonly service: string, public readonly cause?: unknown) {
    super(message);
    this.name = "UpstreamServiceError";
  }
}

export function toToolResult(err: unknown) {
  if (err instanceof ToolInputError) {
    return { isError: true as const, content: [{ type: "text" as const, text: `Invalid input:
${err.message}` }] };
  }
  if (err instanceof UpstreamServiceError) {
    return { isError: true as const, content: [{ type: "text" as const, text: `${err.service} is
temporarily unavailable.` }] };
  }
  return { isError: true as const, content: [{ type: "text" as const, text: "An unexpected error
occurred." }] };
}
```

Structured logging with Pino

Console.log-driven debugging doesn't scale past the first production incident. Pino gives you structured JSON logs, near-zero overhead, and — critically for stdio servers — an easy way to force output to stderr instead of stdout.

SRC/LIB/LOGGER.TS

```
import pino from "pino";
import { env } from "../config/env.js";

export const logger = pino({
  level: env.LOG_LEVEL,
  redact: ["req.headers.authorization", "*.apiKey", "*.password", "*.token"],
}, pino.destination(2)); // fd 2 = stderr, never stdout
```

The `redact` array matters as much as the transport choice. Treat it the same way you'd treat a logging middleware in Express that scrubs `Authorization` headers before anything hits disk or a log aggregator — one missed field here is how API keys end up in a support ticket screenshot months later.

CORRELATING A CALL ACROSS LOG LINES

Attach a request/call ID at the top of the transport handler and thread it through every log line for that call, the same pattern as an Express request-ID middleware:

MIDDLEWARE EXCERPT

```
import { randomUUID } from "node:crypto";

app.post("/mcp", (req, res, next) => {
  req.callId = randomUUID();
  const log = logger.child({ callId: req.callId });
  log.info({ path: req.path }, "incoming MCP request");
  next();
});
```

> **WHAT THE MODEL SEES VS. WHAT YOU LOG**
>
> Keep these deliberately different. The model gets a short, actionable message (Chapter 3's `toToolResult`); your logs get the full stack trace, the call ID, and enough context to actually debug the incident. Conflating the two either leaks internals to the client or leaves you with nothing useful when something breaks at 2am.

// CHAPTER 08 / PART II

Testing & the MCP Inspector

Test tool handlers as plain functions with Vitest, then use the official MCP Inspector to exercise the full protocol surface interactively.

THIS CHAPTER COVERS

- Unit-testing tool handlers
- Mocking upstream dependencies
- The MCP Inspector workflow
- A minimal CI-friendly test suite

CHAPTER 08

Testing & the MCP Inspector

Tool handlers are just functions

Because each tool is registered from a plain exported function (Chapter 2's structure), testing one doesn't require spinning up a transport or a client at all — call the handler directly, the same way you'd unit-test an Express controller function by calling it with a mock `req` / `res` instead of going through the HTTP layer.

SRC/TOOLS/GET-FORECAST.TEST.TS

```
import { describe, it, expect, vi } from "vitest";
import { getForecastHandler } from "./get-forecast.js";
import * as weatherApi from "../lib/weather-api.js";

describe("get_forecast", () => {
  it("returns forecast text for a valid city", async () => {
    vi.spyOn(weatherApi, "fetchForecast").mockResolvedValue("Sunny, 22°C");

    const result = await getForecastHandler({ city: "Lisbon" });

    expect(result.isError).toBeUndefined();
    expect(result.content[0].text).toContain("Sunny");
  });

  it("returns a structured error when the API fails", async () => {
    vi.spyOn(weatherApi, "fetchForecast").mockRejectedValue(new Error("timeout"));

    const result = await getForecastHandler({ city: "Lisbon" });

    expect(result.isError).toBe(true);
    expect(result.content[0].text).not.toContain("timeout"); // no leaked internals
  });
});
```

That second test is doing real work: it asserts the error-redaction behavior from Chapter 7 stays true over time, not just that the function doesn't crash.

The MCP Inspector

The Inspector is the closest thing MCP has to Postman — a standalone UI that connects to your server over stdio or HTTP, lists its tools/resources/prompts, and lets you call them interactively with real request/response inspection. Wire it into a package script so it's a one-command habit, not a separate setup step:

PACKAGE.JSON (EXCERPT)

```
{
  "scripts": {
    "inspect": "npx @modelcontextprotocol/inspector node dist/index.stdio.js",
    "test": "vitest run",
    "test:watch": "vitest"
  }
}
```

`npm run inspect` opens a local web UI at a printed `localhost` URL, spawns your compiled server as a subprocess, and gives you a live tree of every tool with a form generated straight from its Zod schema — the fastest way to sanity-check that a schema change didn't silently break how a tool call form looks.

> **TEST PYRAMID FOR AN MCP SERVER**
>
> **Unit tests** (Vitest) for tool handler logic and edge cases — most of your suite. **Integration tests** for the auth and rate-limit middleware against a real Express app instance. **Manual/exploratory** via the Inspector for protocol-level behavior and new tool discovery. The same pyramid you'd draw for a REST API, with the Inspector standing in for Postman.

// CHAPTER 09 / PART III

Containerizing with Docker

A multi-stage Dockerfile and a docker-compose setup that gets a new contributor from clone to a running server in one command.

THIS CHAPTER COVERS

- Multi-stage builds for a small image
- docker-compose for local dev
- Health checks
- Running stdio vs HTTP in a container

CHAPTER 09

Containerizing with Docker

A multi-stage Dockerfile

Build in one stage with full `devDependencies`, run in a second stage with only production dependencies and compiled output — the same pattern you'd already use for an Express API, just pointed at `dist/index.http.js` instead of a typical server entrypoint.

```
DOCKERFILE
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist

USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD node -e "require('http').get('http://localhost:3000/health', r => process.exit(r.statusCode === 200
? 0 : 1))"

CMD ["node", "dist/index.http.js"]
```

`USER node` matters more here than in a typical hobby project: an MCP server that got compromised through a prompt-injected tool call shouldn't also have root inside its own container.

docker-compose for local development

DOCKER-COMPOSE.YML

```yaml
services:
  mcp-server:
    build: .
    ports:
      - "3000:3000"
    env_file: .env
    depends_on:
      - postgres
    restart: unless-stopped

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: devpassword
      POSTGRES_DB: mcp_dev
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

`docker compose up` should be the entire onboarding instruction for a new contributor — database included. If that single command doesn't get someone to a working server, the README's "clone to first tool call" promise (Chapter 13) doesn't hold up.

> **STDIO DOESN'T NEED A CONTAINER IN DEV**
>
> Docker matters for the HTTP transport, where you're shipping a long-running service. For local stdio usage under Claude Desktop, running the compiled JS directly on the host is simpler and is how the client expects to spawn it anyway — reserve the container for the deployable HTTP variant.

// CHAPTER 10 / PART III

Deployment

Two deploy targets, two different trade-offs: a long-running host for stateful session handling, or a serverless edge platform for pay-per-request economics.

THIS CHAPTER COVERS

- Railway / Fly.io (persistent process)
- Cloudflare Workers / Vercel (serverless)
- Session state across cold starts
- Picking the right target

CHAPTER 10

Deployment: Railway, Fly.io & Serverless

Persistent-process hosting: Railway & Fly.io

For most hosted MCP servers, a long-running Node process is the simpler mental model — the in-memory session map from Chapter 4 just works, the same way it would for any Express app you've deployed to Heroku, Render, or a raw VM before.

FLY.TOML

```
app = "mcp-starter"
primary_region = "iad"

[build]
  dockerfile = "Dockerfile"

[env]
  NODE_ENV = "production"

[[services]]
  internal_port = 3000
  protocol = "tcp"

  [[services.ports]]
    port = 443
    handlers = ["tls", "http"]

  [[services.http_checks]]
    path = "/health"
    interval = "15s"
```

Railway needs even less configuration — point it at the Dockerfile from Chapter 9, set the environment variables from Chapter 2's schema in its dashboard, and it deploys on every push to `main` by default.

Serverless: Cloudflare Workers or Vercel

Serverless suits low, spiky traffic well — you pay per request, not for an idle process — but it changes one assumption the HTTP transport code relied on: nothing survives between invocations, including the in-memory `sessions` map.

SRC/INDEX.WORKER.TS (CLOUDFLARE)

```
import { buildServer } from "./server.js";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/streamableHttp.js";

export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    // Session state moves to Cloudflare KV / Durable Objects —
    // there is no long-lived process to hold a Map in memory.
    const server = buildServer();
    const transport = new StreamableHTTPServerTransport({ sessionIdGenerator: () => crypto.randomUUID()
});
    await server.connect(transport);
    return transport.handleFetchRequest(request);
  },
};
```

TARGET	BEST FOR	WATCH OUT FOR
Railway / Fly.io	Steady traffic, simple session state, WebSocket-like long connections	Pay for idle time; you manage scaling
Cloudflare Workers	Spiky/low traffic, global edge latency	No native TCP/filesystem; session state needs KV or Durable Objects
Vercel Functions	Teams already deployed on Vercel	Execution time limits on longer-running tool calls

SHIP BOTH CONFIGS

A starter kit worth paying for includes both deploy paths pre-wired rather than picking one — the right target depends on the buyer's existing infrastructure, not on the kit author's preference. Let the person deploying choose based on their traffic shape and what they already operate.

// CHAPTER 11 / PART III

CI/CD with GitHub Actions

Lint and test on every push, so a broken tool schema or a failing handler never makes it past a pull request.

THIS CHAPTER COVERS

- A lint + test workflow
- Caching npm installs
- Gating merges on CI status
- Optional: auto-deploy on merge

CHAPTER 11

CI/CD with GitHub Actions

Lint and test on every push

Nothing exotic here — the same workflow shape you'd write for any TypeScript service, run against the same `npm run lint` and `npm test` scripts a contributor runs locally.

```
.GITHUB/WORKFLOWS/CI.YML
name: CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test -- --coverage

      - name: Upload coverage
        uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/
```

Add branch protection requiring this workflow to pass before merge — the CI config is only as useful as the merge rule that enforces it, the same gap that quietly undermines a lot of otherwise well-tested projects.

Optional: deploy on merge

Once Chapter 10's deploy target is chosen, extend the workflow with a second job that runs only on `main` and only after the first job succeeds:

.GITHUB/WORKFLOWS/CI.YML (CONTINUED)

```
  deploy:
    needs: lint-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Fly.io
        uses: superfly/flyctl-actions/setup-flyctl@master
      - run: flyctl deploy --remote-only
        env:
          FLY_API_TOKEN: ${{ secrets.FLY_API_TOKEN }}
```

SECRETS IN CI

Store deploy tokens and API keys as encrypted GitHub Actions secrets, never as workflow-file literals — the same rule from Chapter 2's environment-variable pattern, applied to the pipeline that ships your code instead of the running server itself.

// CHAPTER 12 / PART IV

Real-World Integration Examples

Four tools that demonstrate every pattern from Parts I and II against real dependencies, not toy examples: a database, a REST API, the filesystem, and the web.

THIS CHAPTER COVERS

- Database query tool (Postgres)
- REST API wrapper tool
- Filesystem search/read tool
- Web fetch tool

CHAPTER 12

Real-World Integration Examples

Database query tool

Use a query builder, not raw string concatenation — the SQL-injection discipline you already apply to an Express API applies identically here, arguably with higher stakes since the "input" is model-generated rather than typed by a known user.

SRC/TOOLS/QUERY-ORDERS.TS

```
import { z } from "zod";
import { db } from "../lib/db.js"; // e.g. Kysely or Drizzle

const Input = z.object({
  status: z.enum(["pending", "shipped", "cancelled"]).optional(),
  limit: z.number().int().min(1).max(50).default(10),
});

export function registerQueryOrders(server: McpServer) {
  server.tool("query_orders", "List recent orders, optionally filtered by status", Input.shape,
    async ({ status, limit }) => {
      let query = db.selectFrom("orders").selectAll().orderBy("created_at", "desc").limit(limit);
      if (status) query = query.where("status", "=", status);
      const rows = await query.execute();
      return { content: [{ type: "text", text: JSON.stringify(rows) }] };
    });
}
```

REST API wrapper tool

Wrapping a third-party API is mostly about translating its errors and its own rate limits into the patterns from Chapters 6 and 7, not just proxying a fetch call.

SRC/TOOLS/LOOKUP-SHIPMENT.TS

```
const Input = z.object({ trackingNumber: z.string().min(6) });

export function registerLookupShipment(server: McpServer) {
  server.tool("lookup_shipment", "Get shipment status from the carrier API", Input.shape,
    async ({ trackingNumber }) => {
      const res = await fetch(`https://api.carrier.example/v1/track/${trackingNumber}`, {
        headers: { Authorization: `Bearer ${env.CARRIER_API_KEY}` },
      });
      if (res.status === 429) {
        return { isError: true, content: [{ type: "text", text: "Carrier API rate limit reached, try again
shortly." }] };
      }
      if (!res.ok) {
        return { isError: true, content: [{ type: "text", text: "Could not retrieve shipment status." }]
};
      }
      const data = await res.json();
      return { content: [{ type: "text", text: JSON.stringify(data) }] };
    });
}
```

Filesystem search/read tool

Filesystem tools are the highest-risk category in this chapter — a model with unrestricted filesystem access is a real security surface. Constrain reads to an allow-listed root directory, exactly like you'd sanitize a path parameter in an Express static-file route to prevent directory traversal.

SRC/TOOLS/READ-FILE.TS

```
import path from "node:path";
import { readFile } from "node:fs/promises";

const ALLOWED_ROOT = path.resolve("./workspace");
const Input = z.object({ relativePath: z.string() });

export function registerReadFile(server: McpServer) {
  server.tool("read_file", "Read a text file from the project workspace", Input.shape,
    async ({ relativePath }) => {
      const resolved = path.resolve(ALLOWED_ROOT, relativePath);
      if (!resolved.startsWith(ALLOWED_ROOT)) {
        return { isError: true, content: [{ type: "text", text: "Path escapes the allowed workspace." }]
};
      }
      const contents = await readFile(resolved, "utf-8");
      return { content: [{ type: "text", text: contents }] };
    });
}
```

PATH.RESOLVE ALONE ISN'T ENOUGH

The `startsWith` check after `resolve` is what actually blocks `../../etc/passwd`-style traversal — resolving the path alone doesn't reject it. This is the single most common vulnerability in filesystem-access MCP servers found in the wild.

Web fetch tool

A general-purpose fetch tool needs the same SSRF-conscious guardrails you'd put on any server-side "fetch a URL the user gives you" feature: block internal/private IP ranges, cap response size, and set a timeout.

SRC/TOOLS/FETCH-URL.TS

```
const Input = z.object({ url: z.string().url() });

export function registerFetchUrl(server: McpServer) {
  server.tool("fetch_url", "Fetch and return the text content of a public URL", Input.shape,
    async ({ url }) => {
      const parsed = new URL(url);
      if (isPrivateHostname(parsed.hostname)) {
        return { isError: true, content: [{ type: "text", text: "Refusing to fetch internal/private
addresses." }] };
      }
      const res = await fetch(url, { signal: AbortSignal.timeout(8000) });
      const text = (await res.text()).slice(0, 20_000); // cap response size
      return { content: [{ type: "text", text }] };
    });
}
```

// CHAPTER 13 / PART IV

Developer Experience Polish

The details that make a starter kit feel finished instead of merely functional: a scaffolding CLI, consistent linting, and a README that delivers on a ten-minute promise.

THIS CHAPTER COVERS

- A CLI to scaffold new tools
- Pre-configured linting
- Writing a README that's actually followed
- Versioning & changelogs

CHAPTER 13

Developer Experience Polish

A CLI to scaffold new tools

A small Node script that generates a new tool file, its test file, and registers it automatically removes the most common source of copy-paste drift — the same value a Plop or Hygen generator adds to a large Express codebase's route files.

SCRIPTS/ADD-TOOL.TS

```
import { writeFileSync, readFileSync } from "node:fs";

const name = process.argv[2];
if (!name) {
  console.error("Usage: npm run add-tool -- tool_name");
  process.exit(1);
}

const pascalName = name.replace(/(^|_)(\w)/g, (_, __, c) => c.toUpperCase());

writeFileSync(`src/tools/${name}.ts`, `import { z } from "zod";
import type { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";

const Input = z.object({
  // TODO: define input schema
});

export function register${pascalName}(server: McpServer) {
  server.tool("${name}", "TODO: describe what this tool does", Input.shape,
    async (input) => {
      return { content: [{ type: "text", text: "TODO" }] };
    });
}
`);

console.log(`Created src/tools/${name}.ts — register it in src/tools/index.ts`);
```

PACKAGE.JSON (EXCERPT)

```
{ "scripts": { "add-tool": "tsx scripts/add-tool.ts" } }
```

Pre-configured linting

Biome ships formatting and linting in one fast, zero-config-by-default binary — a lighter alternative to an ESLint + Prettier combo for a starter kit where "clone and go" matters more than deep customizability.

BIOME.JSON

```
{
  "linter": { "enabled": true, "rules": { "recommended": true } },
  "formatter": { "enabled": true, "indentStyle": "space" },
  "organizeImports": { "enabled": true }
}
```

A README that's actually followed

Structure it around the one metric that matters for a starter kit: time from `git clone` to a successful first tool call. Everything else — architecture explanation, deployment options, the full tool catalog — belongs after that path, not before it.

1. **Clone & install** — one code block, copy-pasteable, no prose in between the commands
2. **Configure** — copy `.env.example`, fill in the one required key
3. **Run** — `npm run inspect` to see it working immediately via the Inspector
4. **Add your first tool** — `npm run add-tool -- my_tool`, pointing at the pattern from this chapter

> **TIME IT, DON'T GUESS IT**
>
> Actually run through the README on a clean machine (or a fresh container) and time it before publishing. "Under 10 minutes" as a claim in a product listing needs to survive someone else's terminal, not just your own already- configured environment.

// CHAPTER 14 / PART IV

The Shipping Checklist

Everything from the previous thirteen chapters, compressed into a pre-launch checklist and mapped against the tiers you're likely to sell.

THIS CHAPTER COVERS

- The pre-launch checklist
- Mapping features to pricing tiers
- Distribution channels that work
- What "done" looks like

CHAPTER 14

The Shipping Checklist

Pre-launch checklist

- **Foundation:** strict TypeScript, both transports wired, folder structure matches Chapter 2
- **Infrastructure:** auth, rate limiting, structured errors, and env-validated config all present and tested
- **Tests:** Vitest suite covers every shipped tool's success and error paths; CI is green
- **Docker:** `docker compose up` works on a clean checkout with no manual steps
- **Deploy configs:** at least two targets verified end-to-end, not just written and untested
- **README:** clone-to-first-tool-call timed under the promised window
- **Example tools:** 3-4 real integrations, not placeholder stubs

Mapping features to tiers

If you're packaging this as a sellable product rather than an internal template, the chapters above map cleanly onto a tiered offering — the same logic as splitting a SaaS product into free, pro, and enterprise plans by capability rather than by usage volume.

TIER	CHAPTERS INCLUDED	WHAT IT'S FOR
Starter	1–7 (foundation, auth, rate limiting, errors)	A developer who wants the infrastructure but will write their own tools
Pro	Starter + 8–12 (testing, Docker, deploy, CI, real tools)	A team that wants the full production path plus working examples
Team / License	Pro + commercial license terms + priority support	An agency or company shipping this to clients or across multiple internal projects

Distribution that actually reaches buyers

A free, thin version on GitHub — core scaffold only, no auth or deploy configs — does the credibility-building work; the MCP-specific directories (mcp.so, mcpservers.org, Smithery) put

it in front of people already looking for exactly this, which converts better than general developer audiences who aren't yet building an MCP server.

> **THE UPSELL PATH**
>
> Once this kit is validated, it pairs naturally with a broader Node SaaS boilerplate — auth, billing, dashboard — as a second, related product line rather than a one-off. The infrastructure instincts in this book (env validation, rate limiting, structured errors) are the same instincts that boilerplate needs; you're not starting from zero on the second product either.

// APPENDIX A

Quick-Reference Cheat Sheet

The commands and snippets you'll reach for most often, in one place.

THIS APPENDIX COVERS

- Common CLI commands
- File structure recap
- Tool registration snippet
- Environment variable reference

APPENDIX A

Quick-Reference Cheat Sheet

Commands

COMMAND	WHAT IT DOES
`npm run dev`	Run the server locally with hot reload
`npm run inspect`	Launch the MCP Inspector against the compiled server
`npm run add-tool -- my_tool`	Scaffold a new tool file + registration
`npm test`	Run the Vitest suite once
`npm run lint`	Run Biome lint + format check
`docker compose up`	Start the server + database locally
`flyctl deploy`	Deploy the HTTP variant to Fly.io

Minimal tool registration

```
import { z } from "zod";

server.tool("tool_name", "One-line description for the model",
  { field: z.string().describe("what this field means") },
  async ({ field }) => {
    return { content: [{ type: "text", text: "result" }] };
  }
);
```

Environment variables

VARIABLE	REQUIRED	PURPOSE
`API_KEYS`	Yes	Comma-separated list of valid API keys
`PORT`	No (default 3000)	HTTP transport listen port
`DATABASE_URL`	If using a DB tool	Postgres connection string
`RATE_LIMIT_PER_MINUTE`	No (default 60)	Per-key sliding window limit
`LOG_LEVEL`	No (default info)	Pino log level

// APPENDIX B

Further Reading & Resources

Where to go deeper on each piece of the stack this book touches.

THIS APPENDIX COVERS

- Protocol & SDK docs
- Libraries used in this book
- Deployment platform docs
- Where to distribute a finished kit

APPENDIX B

Further Reading & Resources

PROTOCOL & SDK

- Official Model Context Protocol specification & docs — modelcontextprotocol.io
- TypeScript SDK reference — the `@modelcontextprotocol/sdk` package on npm
- MCP Inspector — the interactive testing tool referenced in Chapter 8

LIBRARIES USED IN THIS BOOK

- **Zod** — schema validation (zod.dev)
- **Vitest** — test runner (vitest.dev)
- **Pino** — structured logging
- **Biome** — linting & formatting

DEPLOYMENT PLATFORMS

- Fly.io and Railway documentation for persistent-process hosting
- Cloudflare Workers and Vercel documentation for serverless deployment

WHERE TO DISTRIBUTE

- mcp.so, mcpservers.org, and Smithery — MCP-specific server directories
- Product Hunt, r/cursor, and Cursor/Claude Code Discord communities
- Gumroad or Lemon Squeezy for selling the packaged Pro/Team tiers